BotXRGame — XR Link Test Guide

Verifying the Galaxy XR to ROS 2 link on a Qualcomm board

For: ROS Dev | Target: RubikPi 3, Ubuntu 24.04, ROS 2 Jazzy | v1.0, 10 Aug 2026

What this is

This guide gets you from a fresh RubikPi 3 to watching live velocity commands from the Samsung Galaxy XR headset arrive as ROS 2 messages. It takes about 20 minutes, and about half of it can be done before the headset is anywhere near you.

Two pieces are involved:

- **ros_tcp_endpoint** — the bridge server. Accepts the headset's TCP connection and republishes what it receives onto the ROS graph. **Use the copy in this repository.** It is patched; upstream does not work.
- **link_monitor** — a diagnostic node that discovers topics at runtime and prints every message it sees, with rate and jitter. It is a plain DDS subscriber, so it sees traffic no matter how it arrived. That is what lets you tell “the message never reached ROS” apart from “the bridge is misconfigured” — the two look identical from the app.

1. Prerequisites

Requirement	Detail
Board	RubikPi 3 running Ubuntu 24.04 LTS
ROS	ROS 2 Jazzy, installed natively (24.04 pairs with Jazzy; no Docker needed)
Network	Board and headset on the same subnet. Wired board + Wi-Fi headset is fine provided the router does not isolate them.
Access	SSH to the board. tmux recommended — you will run two long-lived processes.

Every ROS machine on this project must run **Jazzy**. ROS 2 does not support traffic between distros — a Humble node and a Jazzy node cannot share a DDS graph, and the failure mode is ugly rather than obvious.

2. One-time setup

```
# on the board
git clone https://github.com/swapnil-0/BotXRGame.git
mkdir -p ~/ros2_ws/src
cp -r BotXRGame/ros2_ws/src/ros_tcp_endpoint ~/ros2_ws/src/
cp -r BotXRGame/ros2_ws/src/xr_link_test ~/ros2_ws/src/

cd ~/ros2_ws
colcon build --symlink-install
source install/setup.bash
```

Save yourself repeating the source step in every new shell:

```
echo 'source /opt/ros/jazzy/setup.bash' >> ~/.bashrc
echo '[ -f ~/ros2_ws/install/setup.bash ] && source ~/ros2_ws/install/setup.bash'
>> ~/.bashrc
```

3. Prove it works without the headset

Do this first. It takes two minutes and isolates ROS from the network and the app, so any later failure has a much smaller search space.

```
# terminal 1
python3 ~/ros2_ws/src/xr_link_test/xr_link_test/link_monitor.py \
--ros-args -p preset:=headset -p full_message:=false

# terminal 2
ros2 topic pub -r 10 /cmd_vel geometry_msgs/msg/Twist \
'{linear: {x: 0.25}, angular: {z: -0.5}}'
```

Expected: `>>> FIRST MESSAGE on /cmd_vel <<<` appears immediately, then a steady stream at 10 Hz. On a healthy board, jitter here is under 1 ms — this is your local baseline, and everything wireless gets compared against it.

4. Start the bridge and the monitor

Use tmux so these survive an SSH disconnect. `Ctrl-b c` makes a new window, `Ctrl-b 0/1` switches, `Ctrl-b d` detaches, `Ctrl-b [` scrolls (q to exit scroll).

```
tmux new -s demo

# window 0 — the bridge
ros2 run ros_tcp_endpoint default_server_endpoint --ros-args -p ROS_IP:=0.0.0.0
```

Heads up: Bind to `0.0.0.0`, not the board's own IP. Binding to a specific address makes the socket reachable on one interface only — which silently breaks the moment the board has both Ethernet and Wi-Fi.

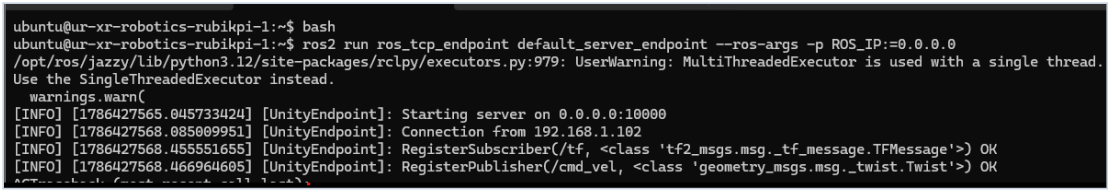

Figure 1 — The bridge, started and with the headset connected. “Starting server on 0.0.0.0:10000” confirms it is listening. The MultiThreadedExecutor warning is harmless. The Connection / RegisterSubscriber / RegisterPublisher lines appear when the headset connects.

This window is your authority on whether the headset is connected.

Line	Meaning
Starting server on 0.0.0.0:10000	Listening. Good.
Connection from 192.168.1.x	The headset's TCP session opened.
RegisterPublisher(/cmd_vel, ...) OK	The app declared its publisher. Data should follow.
Disconnected from ...	Session closed — usually the headset came off the user's face.

A topic can stay visible in `ros2 topic list` after the headset disconnects, because the bridge keeps the publisher node alive. **“Topic exists” does not mean “headset connected.”** Trust this window.

5. Watch the traffic

```
# window 1 (Ctrl-b c)
python3 ~/ros2_ws/src/xr_link_test/xr_link_test/link_monitor.py \
--ros-args -p preset:=headset -p full_message:=false
```

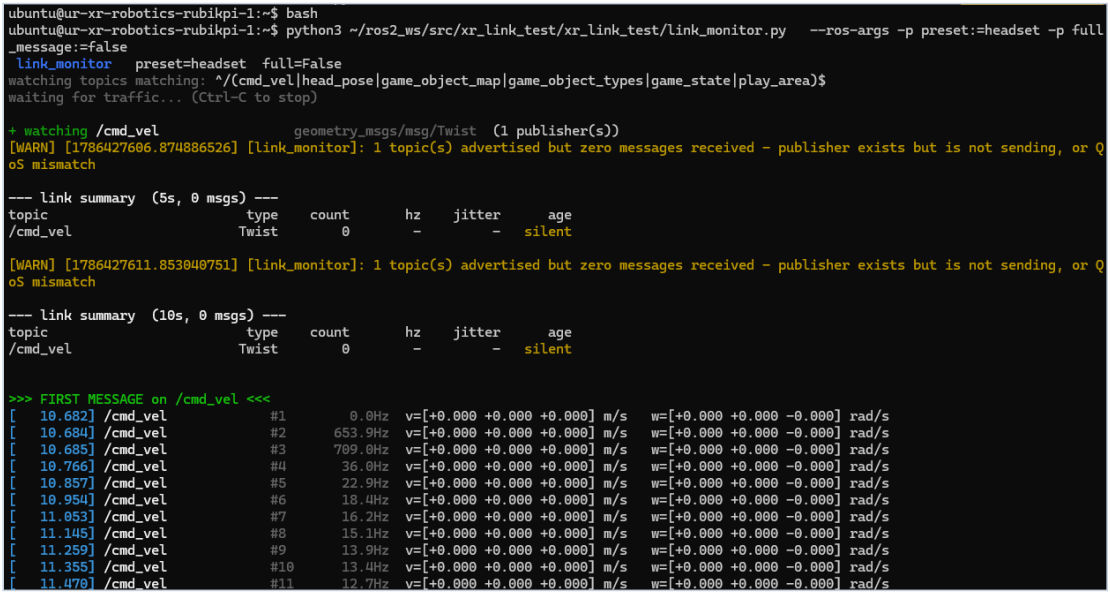

Figure 2 — Startup and first contact. The repeating warning while nothing is publishing is correct: it means the monitor is alive and watching. “+ watching /cmd_vel” appears when the topic is registered, and FIRST MESSAGE marks the moment real data arrives. Ignore the wild Hz values on the first few messages — the rate average needs a few samples to settle.

Now have the user put the headset on, enter the board's IP and port 10000, press **Connect** (not Skip), and move the thumbstick.

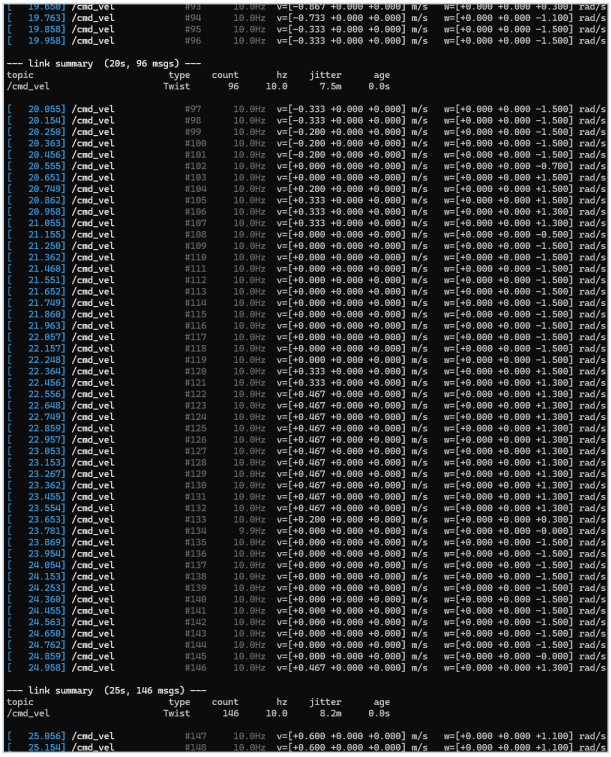

Figure 3 — A healthy link. Velocity values track the thumbstick, the rate holds steady, and the summary table every 5 seconds gives count, rate, jitter and age per topic.

6. Reading the numbers

Column	Meaning	Healthy
count	Messages received since start	Rising steadily
hz	Rolling publish rate	Matches the app's configured rate
jitter	Std deviation of inter-arrival gaps	Single-digit ms
age	Seconds since the last message	Under ~0.2 s while driving

Reference measurements, Galaxy XR to RubikPi 3 over Wi-Fi, 10 Aug 2026:

Scenario	Rate	Jitter
Local ros2 topic pub (no network)	10.0 Hz	0.9 ms
Headset over Wi-Fi	10.0 Hz	3–8 ms

If jitter is far above that, suspect the radio before the code: 2.4 GHz congestion, a busy channel, or Wi-Fi power saving on either device. Some jitter also comes from the app quantising its publish timer to the headset's render loop, which is expected.

7. Troubleshooting

Work outward from ROS toward the headset. Each row isolates one layer.

Symptom	What it means / what to do
Monitor never shows any topic	Nothing is publishing. Run the section 3 test to confirm ROS and the monitor are fine.
Headset app retries forever, bridge logs nothing	Traffic is not reaching the board. Check with: sudo tcpdump -i any -n 'tcp port 10000'. SYN with an RST reply means nothing is listening — the bridge is not running.
Bridge shows Connection then Disconnected in a loop	Protocol mismatch. Confirm you are running the patched copy from this repo, not a fresh upstream clone.
Publisher registers, but zero messages arrive	Restart the bridge with -p debug_frames:=true to log every inbound frame. Frames present means the bridge is dropping them; only __ commands means the app is not sending.
Connection drops after ~20 seconds	Normal. The headset was removed, so the app paused and closed the socket.
ros2: command not found	The shell did not source ROS. Run bash, or source the two setup files.

8. Reference

link_monitor parameters

Parameter	Default	Purpose
preset	all	headset | robot | all — preset topic filters
include	(unset)	Regex; overrides preset when set
exclude	noise topics	Regex of topics to skip
full_message	true	Print full YAML body for unrecognised types
stats_period	5.0	Seconds between summary tables; 0 disables

Topics, headset to robot

Topic	Type	Notes
/cmd_vel	geometry_msgs/Twist	linear.x forward, angular.z turn
/head_pose	geometry_msgs/PoseStamped	Headset pose, 30 Hz
/game_state	std_msgs/String	SETUP | PHASE1_PLAYER | PHASE2_AUTO | GAME_OVER
/game_object_map	geometry_msgs/PoseArray	Virtual object positions

Full topic contract: `docs/ros-interface/ros-unity-interface.md`. Why the bridge is patched: `ros2_ws/src/ros_tcp_endpoint/PATCHES.md`.

Note: the ROS-TCP-Connector bridge is a stopgap. A BotXRGame-owned transport is planned, with a local gateway publishing into ROS. Topic names and semantics above are expected to survive that change; the transport underneath is not.